Three Number Sum

AlgoExpert · Medium · Arrays

The problem

- All triplets of distinct integers summing to target, each triplet ascending and the list sorted.
- Sort then sweep with two pointers.

Intuition

- Fix one number, then find pairs in the sorted remainder with two pointers.

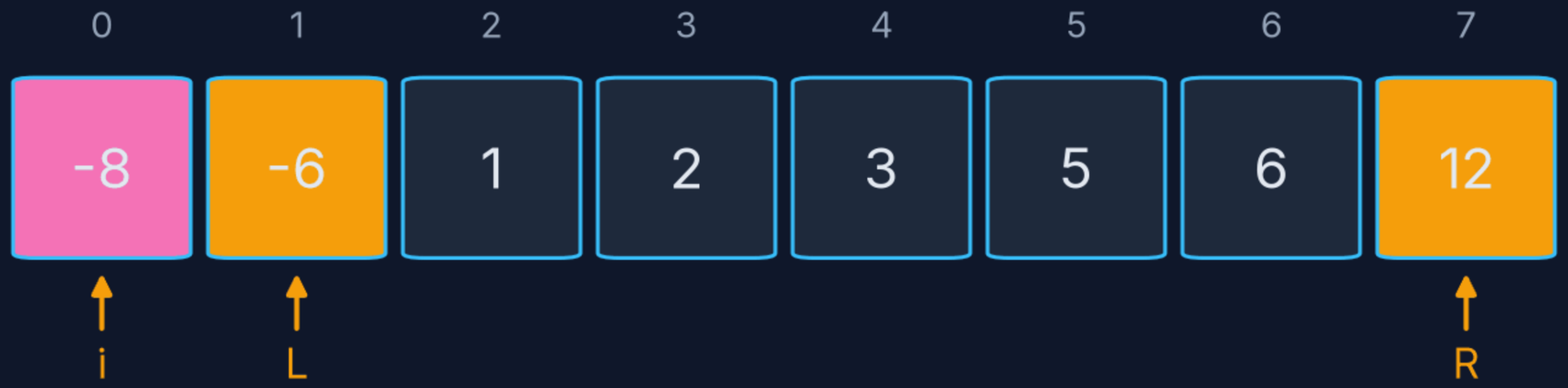
Brute force vs optimal

- Brute force: three nested loops → $O(n^3)$
- Fix one + hash set for the pair → $O(n^2)$ time, $O(n)$ space
- Sort + fix one + two pointers → $O(n^2)$, $O(1)$ extra

Approach

- 1. Sort the array
- 2. For each index, set left and right pointers after it
- 3. Move them based on the current sum
- 4. Record exact matches

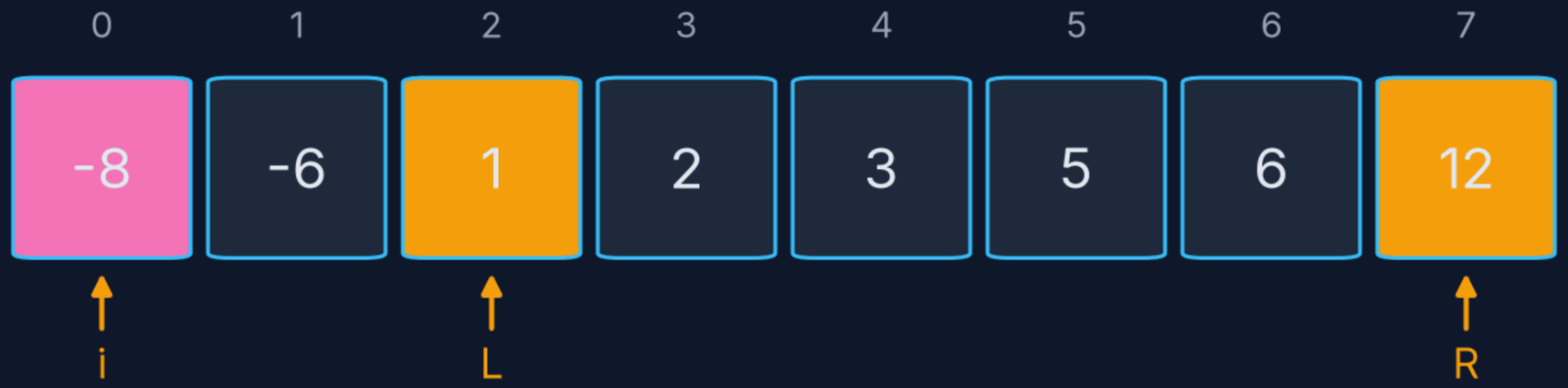
Worked example




```
-8 + -6 + 12 = -2
triplets: []
```

sum -2 < 0 → L++

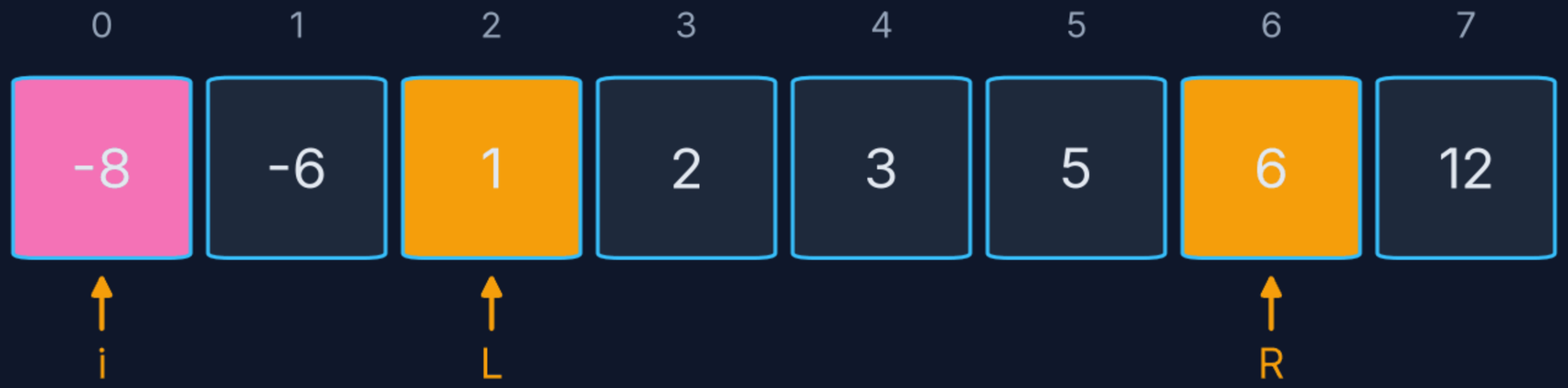
Worked example




```
-8 + 1 + 12 = 5
triplets: []
```

sum 5 > 0 → R−−

Worked example




```
-8 + 1 + 6 = -1
triplets: []
```

sum -1 < 0 → L++

Worked example




```
-8 + 2 + 6 = 0
triplets: [[-8, 2, 6]]
```

sum 0 = 0 ✓

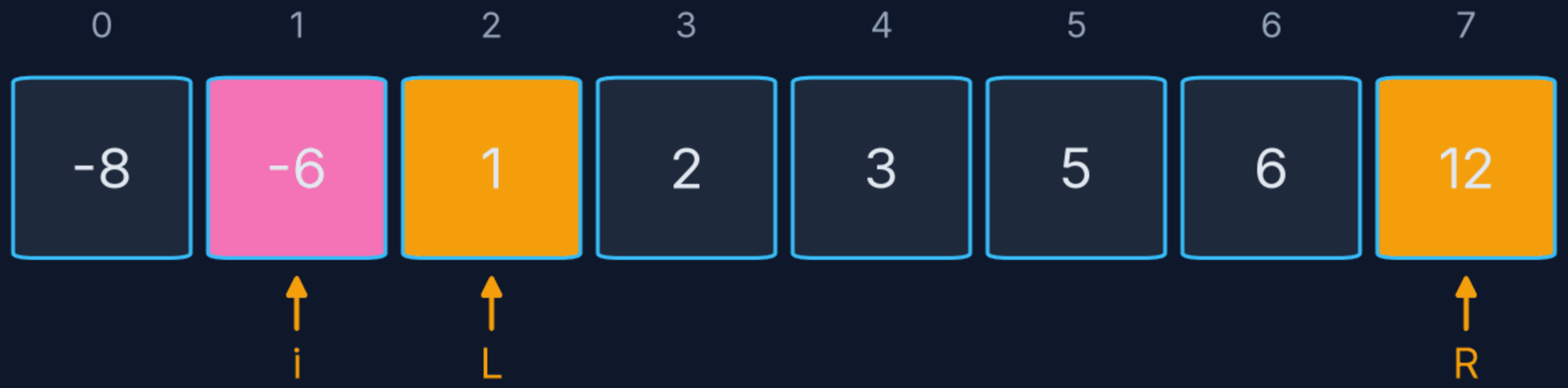
Worked example




```
-8 + 3 + 5 = 0
triplets: [[-8, 2, 6], [-8, 3, 5]]
```

sum 0 = 0 ✓

Worked example




```
-6 + 1 + 12 = 7
triplets: [[-8, 2, 6], [-8, 3, 5]]
```

sum 7 > 0 → R−−

Java solution

```java
public class ThreeNumberSum {
    public static List<Integer[]> threeNumberSum(int[] array, int target) {
        Arrays.sort(array);
        List<Integer[]> out = new ArrayList<>();
        for (int i = 0; i < array.length - 2; i++) {
            int l = i + 1, r = array.length - 1;
            while (l < r) {
                int s = array[i] + array[l] + array[r];
                if (s == target) { out.add(new Integer[]{array[i], array[l], array[r]}); l++; r--; }
                else if (s < target) l++; else r--;
            }
        }
        return out;
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Edge cases & pitfalls

- Distinct values here; with duplicates, skip repeats
- Move both pointers after a match
- Fewer than 3 elements → empty

Interview tips & follow-ups

- Mention generalisation to k-sum
- Follow-up: closest 3-sum

Recap

- Fix one number, then find pairs in the sorted remainder with two pointers.
- Complexity: O(n^2)